

HTable : Liste des fonctionnalités (côté utilisateur) et pipeline

NB : Le composant est encore encours de traitement de 4 de bug visuels mineur.

Structure (lignes / colonnes)

- Insérer une ligne au-dessus / en dessous d'une ligne existante.
- Insérer une colonne à gauche / à droite d'une colonne existante (en conservant les largeurs).
- Supprimer une ligne ou plusieurs lignes.
- Supprimer une colonne ou plusieurs colonnes.
- Déplacer une ligne (moveRow).
- Échanger deux lignes (swapRows).
- Vider le tableau (supprimer toutes les lignes, conserver les colonnes).
- Fractionner le tableau en deux (splitTable) à partir d'une ligne.

Fusions & subdivisions

- Fusionner une zone rectangulaire de cellules (mergeCells / mergeSelection).
- Défuser une cellule fusionnée (unmergeCell).
- Fractionner une cellule fusionnée en un maillage targetRows × targetCols (splitCell).
- Subdiviser une cellule en une grille de sous-cellules (splitCellGrid).
- Subdiviser localement une cellule en deux sous-cellules (splitCellLocally / splitCellDirectly).
- Supprimer une subdivision interne (removeInternalGrid / removeInternalGridFromCell).
- Gérer l'intégrité des fusions lors d'opérations structurales (tri, déplacement, insertion/suppression).

Sélection & navigation

- Sélectionner une cellule, une plage rectangulaire, une ligne ou une colonne.
- Sélectionner tout le tableau.
- Récupérer les lignes/colonnes sélectionnées.

- Effacer la sélection.
- Gérer une sélection de cellules multi-colonnes comme dans Word (sélection par header).

Édition de contenu

- Éditer le contenu d'une cellule (édition normale).
- Éditer une sous-cellule interne via un éditeur intégré (internal editor).
- Démarrer/arrêter l'édition du nom d'un en-tête de colonne (header editor).
- Édition par double-clic / validation par Enter / perte de focus.

Formatage visuel (cellules, lignes, colonnes, en-têtes)

- Définir la couleur de fond et de texte d'une cellule, colonne, ligne ou sélection.
- Appliquer des styles prédéfinis au tableau ou uniquement à la sélection (HTableStyle).
- Activer/désactiver l'en-tête stylisée et la ligne « totale ».
- Activer alternance de couleurs sur lignes ou colonnes (banded rows/columns).
- Mettre en valeur la première/dernière colonne.
- Gérer les styles d'en-tête par colonne (couleur, alignement, gras/italique, taille).
- Définir marges internes (padding) pour une cellule ou globalement.
- Définir alignement horizontal/vertical d'une cellule, ligne, colonne, sélection ou global.
- Définir direction du texte (horizontal / vertical up / vertical down).
- Réinitialiser le formatage d'une cellule ou d'une sélection (reset).

Bordures / quadrillage

- Appliquer des bordures sur un ou plusieurs côtés d'une cellule.
- Appliquer la même bordure sur tous les côtés d'une cellule.
- Appliquer une bordure à toutes les cellules (bordure générale).
- Appliquer bordure extérieure (contour du tableau).
- Appliquer bordures internes (séparations internes).
- Appliquer des bordures à la sélection.
- Supprimer une bordure (côté spécifique / toutes).

Données & formules

- Insérer une formule dans une cellule (doit commencer par '='), stocker la formule.
- Évaluer une formule pour une cellule (evaluateFormula).
- Recalculer toutes les formules du tableau (recalculateAllFormulas).
- Exporter le contenu du tableau en texte avec séparateur (convertToText).
- Définir la classe d'une colonne (setColumnClass) pour render/editor.

Tri & ordre

- Trier par une colonne (sortByColumn) ou par plusieurs colonnes (sortByColumns) via un sorter personnalisé.
- Effacer tout tri (clearSort).
- Empêcher le tri natif au clic sur l'en-tête (comportement contrôlé).

Taille & disposition

- Définir hauteur d'une ligne individuelle ou de toutes les lignes.
- Définir largeur d'une colonne individuelle ou de toutes les colonnes.
- Répartir uniformément hauteurs des lignes ou largeurs des colonnes (distributeRowsEvenly / distributeColumnsEvenly).
- Ajuster automatiquement les colonnes selon le contenu ou la fenêtre (autoFit).

Interaction & modes

- Activer/désactiver l'affichage du quadrillage léger (gridVisible).
- Changer le mode d'interaction (normal / draw / erase) et ajuster le curseur en conséquence.
- Redimensionner visuellement lignes/colonnes (et afficher prévisualisation).

Menus & actions

- Ouvrir le menu contextuel sur cellule ou en-tête (showContextMenu / showHeaderMenu).
- Ajouter / supprimer / lister des actions contextuelles personnalisées (ContextAction).
- Ajouter / supprimer / lister des actions d'en-tête (HeaderAction).

- Ouvrir les dialogues utilitaires : dialog de subdivision (SplitCellDialog), dialog de formule (HFormulaDialog).

Rendu & UI

- Déléguer rendu et gestion des interactions UI à HBasicTableUI.
- Mettre à jour immédiatement la présentation via refreshUI (repaint + revalidate).
- Calculer preferred/min/max sizes et gérer l'impression (répéter en-tête).

Sécurité / contraintes utilisateur

- Les cellules absorbées (parties d'une fusion) ne sont pas éditables.
- Vérifications d'indices et protections contre opérations hors bornes.
- Invariants maintenus lors d'insertion/suppression (mergeModel mis à jour).

Utilitaires divers

- Résoudre la cellule sous un point souris (resolvePoint) en tenant compte des fusions.
- API pour obtenir/positionner focus/surlignement/hover interne (focusedRow/hoveredRow/etc.).
- Exposition d'un modèle de sélection cellulaire (HCellSelectionModel) pour usage externe.

PipeLine pour chaque Fonctionnalités :

1. Insérer une ligne au-dessus / en-dessous

- Méthodes : `insertRowAbove(int row)`, `insertRowBelow(int row)`
- Pipeline :
 - i. Validation des indices (borne).
 - ii. Appel vers `hModel.insertEmptyRow(row)` (ou `insertRowAbove` utilise `insertEmptyRow`).
 - iii. Le modèle (`HDefaultTableModel`) met à jour sa grille et `MergeModel` si nécessaire.
 - iv. `HTable` appelle `refreshUI()` (revalidate + repaint).

2. Insérer une colonne à gauche / droite (avec préservation layout)

- Méthodes : `insertColumnLeft(int col)`, `insertColumnLeft(int col, String)`, `insertColumnRight(int col)`, `insertColumnRight(int col, String)`, `insertColumnWithLayout(int insertAt, String, int widthSourceCol)`
- Pipeline :
 - i. Validation d'index.
 - ii. Sauvegarde des largeurs/heights existants (`savedWidths` / `savedHeights`).
 - iii. Délégation : `hModel.insertColumn(insertAt, columnName)`.
 - iv. Ajustement des `preferredWidth/width` des `TableColumn` en réappliquant `savedWidths` (ou `widthSource`).
 - v. Restauration des `rowHeights` sauvegardés.
 - vi. `refreshUI()`.

3. Supprimer une ligne ou plusieurs lignes

- Méthodes : `deleteRow(int)`, `deleteRows(int[])`, `deleteSelectedRows()`
- Pipeline :
 - i. Validation d'index (chaque ligne).
 - ii. Pour suppression multiple : tri inverse et suppression de bas en haut.

- iii. Appel `hModel.removeRow(row)` pour chaque ligne — le modèle défusionne/décale sa grille et met à jour `MergeModel`.
- iv. Après suppression, `refreshUI()`.

4. Supprimer une colonne ou plusieurs colonnes

- Méthodes : `deleteColumn(int)`, `deleteColumns(int[])`
- Pipeline :
 - i. Validation d'index.
 - ii. Avant suppression : détection/défusion des fusions impliquant la colonne (`hModel.defuseColumn()` appelé dans `removeColumn`).
 - iii. `hModel.removeColumn(col)` (rebuild `dataVector`, `resizeGrid`, update `MergeModel`).
 - iv. Ajustement UI (`preferred widths` éventuellement).
 - v. `refreshUI()`.

5. Déplacer / échanger des lignes

- Méthodes : `moveRow(int fromIndex, int toIndex)`, `swapRows(int row1, int row2)`
- Pipeline :
 - i. Validation des indices.
 - ii. Sauvegarde de la ligne source (`Cell[]` copy) et des données (`Vector`) dans le modèle.
 - iii. Déplace/réorganise les vecteurs de données et la grille `Cell[][]` dans `HDefaultTableModel`.
 - iv. `fireTableRowsUpdated()` / `refreshUI()` (mise à jour visible).

6. Fractionner le tableau (`splitTable`)

- Méthode : `splitTable(int atRow)`
- Pipeline :
 - i. Validation `atRow`.
 - ii. Copie des en-têtes et des données des lignes à détacher (`newData`).
 - iii. Suppression des lignes de l'ancien tableau via `hModel.removeRow(r)`.

- iv. Création d'un nouveau HTable(newData, colNames), application du même tableStyle.
- v. refreshUI(); retourne le nouveau HTable.

7. Sélectionner cellule / plage / ligne / colonne / tout

- Méthodes : selectCell(int,int), selectRow(int), selectColumn(int), selectColumns(int,int), selectAll(), setSelection(CellRange)
- Pipeline :
 - i. Validation d'indices (selon type).
 - ii. Mise à jour des ListSelectionModel (row & column selection models) ou usage de HCellSelectionModel pour sélection cellulaire.
 - iii. Mise à jour d'états internes (focusedRow/Column / cellSelectionModel).
 - iv. refreshUI().

8. Éditer le contenu d'une cellule / sous-cellule / header

- Méthodes : startInternalEdit(InternalCellHit), stopInternalEdit(), startHeaderEdit(int), stopHeaderEdit()
- Pipeline internal edit :
 - i. Validation du hit (non null).
 - ii. Calcul des bounds (hit.bounds) et positionnement de internalEditor.
 - iii. Remplissage du texte depuis hit.cell.value et setVisible(true), requestFocus, selectAll.
 - iv. À stopInternalEdit : si editingInternalCell != null, écriture internalEditor.getText() → editingInternalCell.cell.value ; hide editor ; editingInternalCell = null ; repaint().
- Pipeline header edit :
 - i. Vérifie et place headerEditor sur JTableHeader à header.getHeaderRect(colIndex).
 - ii. headerEditor.setText(getColumnName(colIndex)), visible, focus.
 - iii. stopHeaderEdit : lecture headerEditor.getText(), mise à jour TableColumnModel.getColumn(editingColumnIndex).setHeaderValue(newName), cacher éditeur, repaint header.

9. Appliquer couleur / style aux cellules / lignes / colonnes / sélection

- Méthodes : `setCellBackground(row,col,Color)`, `setRowBackground(int,Color)`, `setColumnBackground(int,Color)`, `setSelectionCellBackground(Color)`, `applyBackgroundToSelection(Color)`, `setCellForeground`, `setRowForeground`, `setColumnForeground`, `applyStyleToSelection(HTableStyle)`
- Pipeline :
 - i. Pour opérations sur sélection : si `hasInternalFocus()` → appliquer au `focusedInternalCell.cell.style` directement + repaint ; sinon itérer la sélection (`CellRange`) et appeler `hModel.setCellBackground/getCellForeground` selon cible.
 - ii. `hModel` met à jour `HCellModel.style` pour chaque cellule.
 - iii. `refreshUI()`.

10. Bordures (appliquer / supprimer)

- Méthodes : `setCellBorderSide(...)`, `setCellBorderAll(...)`, `setBorderAll(...)`, `setBorderOuter(...)`, `setBorderInner(...)`, `setSelectionBorder(...)`, `removeCellBorder(...)`, `removeCellBorderSide(...)`
- Pipeline :
 - i. Validation / calcul des cellules cibles (itération si zone ou tout le tableau).
 - ii. Appel `hModel.setCellBorderSide(row,col,sides,color,thickness,style)` pour chaque cellule cible (ou `m.setBorderTop...` si internal focus).
 - iii. `refreshUI()`.

11. Alignement / marges / direction texte

- Méthodes : `setCellAlignment`, `setRowAlignment`, `setColumnAlignment`, `setSelectionAlignment`, `setTableAlignment`, `setCellTextDirection`, `setColumnTextDirection`, `setRowTextDirection`, `setCellMargins`, `setDefaultCellMargins`
- Pipeline :
 - i. Vérification sélection / internal focus.
 - ii. Délégation aux méthodes d'accès du modèle : `hModel.setCellAlignment`, `hModel.setCellTextDirection`, `hModel.setCellMargins`.
 - iii. `refreshUI()`.

12.Fusionner / défusionner / fractionner cellules

- Méthodes (HTable) : `mergeSelection()`, `mergeCells(int r1,int c1,int r2,int c2)`, `unmergeCell(int,row,int col)`, `splitCell(int,row,col,targetRows,targetCols)`, `splitCellGrid(...)`, `splitCellLocally(...)`, `removeInternalGrid(...)`
- Pipeline `mergeSelection` → `mergeCells` :
 - i. Récupère `CellRange sel = getSelection();` si null ou `singleCell` → return.
 - ii. Test `HTableStructureIntegrity.isMergeableViewRange(this, sel...)` — rejette si non mergeable.
 - iii. Convertit coords vue → modèle : `toModelRow` / `toModelColumn`.
 - iv. Test `canMergeSelection` via `hModel.getMergeModel().canMergeSelection(...)` (vérifie chevauchements).
 - v. Appel `mergeCells(sel...)` → `HTable.mergeCells` délègue `hModel.mergeCells(toModelRow(...), toModelColumn(...), ...)`.
 - vi. `HDefaultTableModel.mergeCells` gère sauvegarde des valeurs, création de `mergedValues`, vidage des cellules absorbées et `syncAllSpanFields()`.
 - vii. `refreshUI()`.
- Pipeline `unmergeCell` :
 - i. Convertit vue→modèle.
 - ii. `hModel.unmergeCell(modelRow, modelCol)` qui restaure values et `syncAllSpanFields`.
 - iii. `refreshUI()`.
- Pipeline `splitCell` / `splitCellGrid` / `splitCellLocally` :
 - i. Validation / conversion index si nécessaire.
 - ii. Délégation vers `hModel.splitCell` / `hModel.splitCellGrid` / `hModel.splitCellLocally` (ou `splitCellDirectly` pour `Cell`).
 - iii. `hModel` met à jour `MergeModel` et/ou `InternalGrid`, restaure ou répartit `mergedValues`, met à jour cellule(s).
 - iv. `refreshUI()` ou `fireTableRowsUpdated` selon l'opération.

13. Insérer / supprimer subdivision interne (InternalGrid)

- Méthodes : `splitCellGrid`, `splitCellLocally`, `splitCellDirectly` (via `hModel`), `removeInternalGrid`, `removeInternalGridFromFocused`, `removeInternalGridFromCell`
- Pipeline :
 - i. Validation.
 - ii. Si focus interne, appeler méthode *Directly* sur `Cell` ; sinon appeler `hModel.splitCellGrid(row,col,...)`.
 - iii. `InternalGrid` construit ou supprimé, valeur finale reportée dans `cell.value`.
 - iv. `fireTableRowsUpdated` / `fireTableDataChanged` / `refreshUI()`.

14. Insérer formule, évaluer, recalculer

- Méthodes : `setCellFormula(int row,int col,String formula)`, `evaluateFormula(int row,int col)`, `recalculateAllFormulas()`
- Pipeline `setCellFormula` :
 - i. Vérification que `formula != null` et commence par `"=`".
 - ii. `hModel.getCellModel(row,col).setFormula(formula)` — stockage de la formule dans `HCellModel`.
 - iii. `Object result = HTableFormula.evaluate(formula, hModel, row, col)`.
 - iv. `hModel.setValueAt(result, row, col)` (`DefaultTableModel` API).
 - v. `refreshUI()`.
- Pipeline `recalculateAllFormulas` :
 - i. Itération sur toutes les cellules, lecture de formule via `hModel.getCellModel(r,c).getFormula()`.
 - ii. Pour chaque formule non vide : `result = HTableFormula.evaluate(...)` ; `hModel.setValueAt(result,...)`.
 - iii. `refreshUI()`.

15.Trier par colonne / colonnes

- Méthodes : `sortByColumn(int col, SortOrder order)`, `sortByColumns(int[] cols, SortOrder[] orders)`, `setRowSorter(RowSorter<? extends TableModel>)`
- Pipeline `sortByColumn(s)`:
 - i. Validation indices.
 - ii. Création d'un `TableRowSorter<HDefaultTableModel>` via `newNonClickableSorter()` (surcharge pour `isSortable=false`).
 - iii. `setRowSorter(sorter)` ; construction des `SortKey(s)` et `sorter.setSortKeys(keys)`; `sorter.sort()`.
 - iv. `setRowSorter` ajoute listener qui, en cas de changement d'ordre, appelle `HTableStructureIntegrity.invalidateAllMergesOnSort(this)` puis `refreshUI()`.

16.Ajustement automatique / distribution tailles

- Méthodes : `autoFit(int mode)`, `distributeRowsEvenly()`, `distributeColumnsEvenly()`, `setRowHeight(row,height)`, `setColumnWidth(col,width)`, `setAllRowsHeight`, `setAllColumnsWidth`
- Pipeline `autoFit` :
 - `AUTOFIT_WINDOW` :
 - a. Mesure largeur disponible du parent.
 - b. calcule `colW = availableWidth / getColumnCount()` ; `setAllColumnsWidth(colW)`.
 - `AUTOFIT_CONTENT` :
 - a. Pour chaque colonne : mesurer `preferredSize` de header (`headerRenderer`) et chaque cell renderer (`prepareRenderer`).
 - b. `setPreferredWidth(maxW + 20)`.
- ii. `revalidate()`; `repaint()`.

17.Interaction modes / curseurs / grid visible

- Méthodes : setInteractionMode(int), getInteractionMode(), setGridVisible(boolean), isGridVisible()
- Pipeline :
 - i. Modifier interactionMode ; changer curseur selon mode (CROSSHAIR, HAND, default).
 - ii. refreshUI().

18.Menu contextuel & header actions

- Méthodes : addAction(ContextAction), removeContextAction, clearContextActions, getContextActions, addHeaderAction, removeHeaderAction, clearHeaderActions, getHeaderActions, showContextMenu(TableContext,int,int), showHeaderMenu(HeaderContext,int,int), showSplitCellDialog(int,int), showFormulaDialog(int,int)
- Pipeline ajouter action :
 - i. Délégation menuController.addAction(action).
- Pipeline affichage menu (showContextMenu) :
 - i. Construction du TableContext (par appel du menuController).
 - ii. menuController.showContextMenu(ctx,x,y) : construction du JPopupMenu, filtrage des actions via isVisible/isEnabled, affichage.
 - iii. Exécution d'une action appelle son perform(TableContext).
 - iv. refreshUI() si action modifie le modèle/UI.

19.Rendu / UI / sélection handler

- Méthodes / points : setUI(new HBasicTableUI()), selectAll() delegates to ((HBasicTableUI)getUI()).getSelectionHandler().selectAll()
- Pipeline général rendu :
 - i. HBasicTableUI consulte HDefaultTableModel et HCellModel pour dessiner.
 - ii. HTable met à jour l'état (highlight/hover/focus) et appelle refreshUI(); HBasicTableUI réagit lors du repaint et dessine selon tableStyle et HCellModel.

20. Résolution de point → cellule (hit testing tenant compte des fusions)

- Méthode : `resolvePoint(Point)`
- Pipeline :
 - i. `rowAtPoint(point)`, `columnAtPoint(point)` pour coords vue.
 - ii. Convertir vue → modèle via `toModelRow/toModelColumn`.
 - iii. Si `hModel.isAbsorbed(modelRow, modelCol) → hModel.findMergeOrigin(modelRow, modelCol)` retourne origin point.
 - iv. Convertir origine modèle → vue via `convertRowIndexToView / convertColumnIndexToView` ; renvoyer coords vue de la cellule principale.

21. Exporter en texte

- Méthode : `convertToText(String delimiter)`
- Pipeline :
 - i. Itère en-têtes puis toutes les cellules ; lit `hModel.getValueAt(r,c)` pour chaque cellule.
 - ii. Construit et retourne String (delimiter séparateur).

22. Sécurité / validations / invariants

- Méthodes / checks : `isCellEditable` override (teste `hModel.isAbsorbed`), nombreuses validations index dans chaque méthode.
- Pipeline :
 - i. Avant action : vérification borne/condition (ex. `nbRows>0`, `nbCols>0`, cellule non absorbée).
 - ii. Si invalidité : return (parfois log).
 - iii. Sinon délégation au modèle / UI et `refreshUI()`.